Project - Low Level Design

on

Dockerized Healthcare Python Flask Service

Course Name: DevOps

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name & Enrolment Numbers

Sr no	Student Name	Enrolment Number
1.	Janhavi Gupta	EN22CS301454
2.	Harshita Songara	EN22CS301411
3.	Hariom Patidar	EN22CS301383
4.	Hemant Mandloi	EN22CS301422

Group Name: Group 06D4

Project Number: DO-42

Industry Mentor Name:

University Mentor Name: Prof. Avnesh Joshi

Academic Year: 2025-2026

Table of Contents

1. Introduction.
   1.1. Scope of the document.
   1.2. Intended Audience
   1.3. System overview.

2. System Design.
   2.1. Application Design
   2.2. Process Flow.
   2.3. Information Flow.
   2.4. Class Diagram
   2.5. Components Design
   2.6. Key Design Considerations

3. References

1. Introduction

1.1. Scope of the Document

The purpose of this Software Design Document (SDD) is to provide a detailed low-level design of the Dockerized Healthcare Python Flask Service. It describes the system architecture, design of RESTful APIs using Flask, and the process of containerizing the application using Docker. The document also outlines the configuration of Dockerfile and Docker Compose, the data flow between components, and the deployment process using DevOps practices such as CI/CD pipelines. Additionally, it covers important aspects like error handling, logging, performance, and scalability considerations. This document serves as a reference for developers, DevOps engineers, testers, and stakeholders to understand, develop, deploy, and maintain the system efficiently.

1.2. Intended Audience

Audience	Purpose
Developers	Understand and implement the Flask-based API structure and application logic
DevOps Engineers	Manage Docker containerization, deployment, and CI/CD pipelines
Testers	Validate API functionality, performance, and reliability
Healthcare System Integrators	Integrate the service with hospital or healthcare systems
System Administrators	Monitor, maintain, and manage deployed containers and services

1.3. System Overview

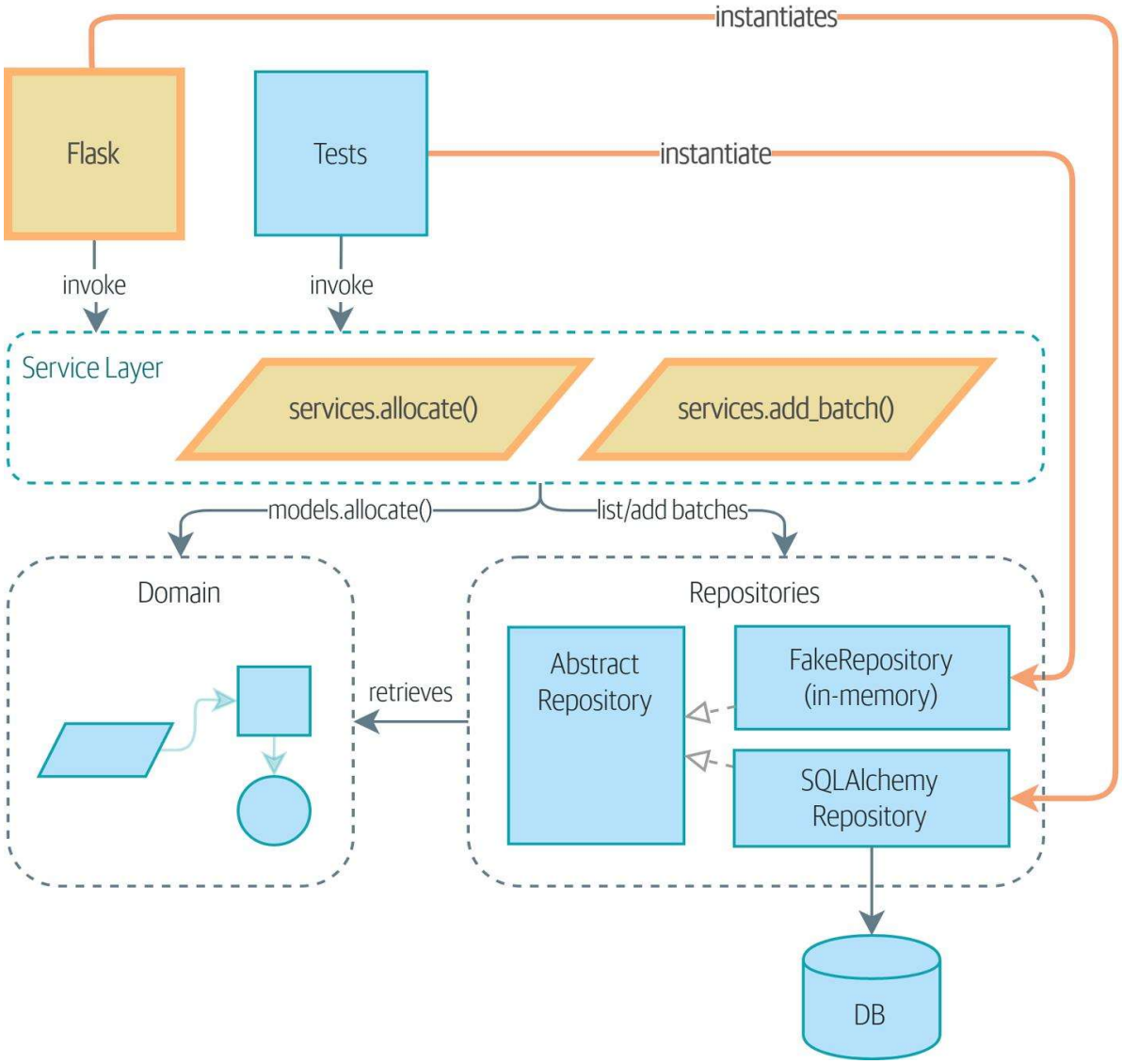
The Dockerized Healthcare Python Flask Service is a backend application designed to provide healthcare-related functionalities through RESTful APIs. It allows users to send requests such as symptoms or health-related queries and receive structured responses including possible disease predictions, treatment suggestions, and doctor recommendations. The system is built using the Flask framework for lightweight and efficient API development and is containerized using Docker to ensure portability, consistency, and easy deployment across different environments. By following DevOps practices, the application supports continuous integration and continuous deployment (CI/CD), enabling faster updates and reliable performance. Overall, the system aims to deliver a scalable, maintainable, and efficient healthcare service that can be easily integrated with other applications or platforms.

Core Capabilities:

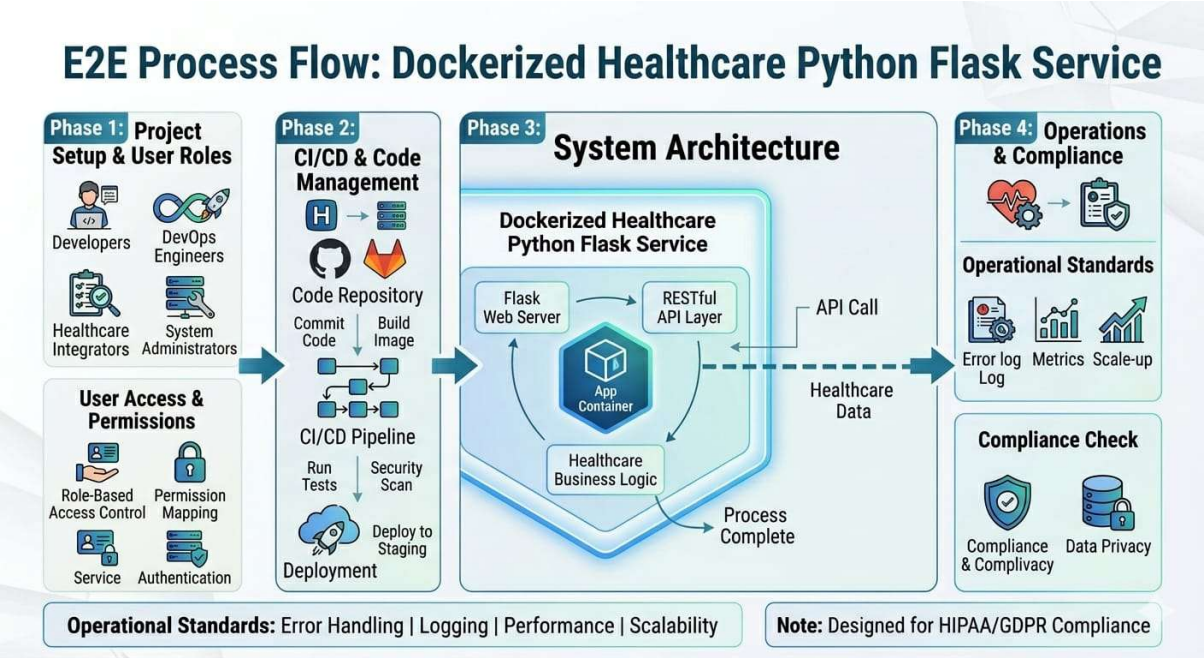
• **Symptom Analysis API:** Accepts user symptoms as input and processes them for further analysis
• **Disease Prediction API:** Identifies possible medical conditions based on provided symptoms
• **Treatment Recommendation API:** Generates basic treatment suggestions and medical guidance
• **Remedy Suggestions API:** Provides home remedies and self-care advice
• **Doctor Recommendation API:** Suggests relevant specialists based on the predicted condition
• **Hospital Locator API:** Helps identify nearby hospitals or healthcare facilities
• **Health Status API:** Returns structured health reports in JSON format for easy integration

2. System Design

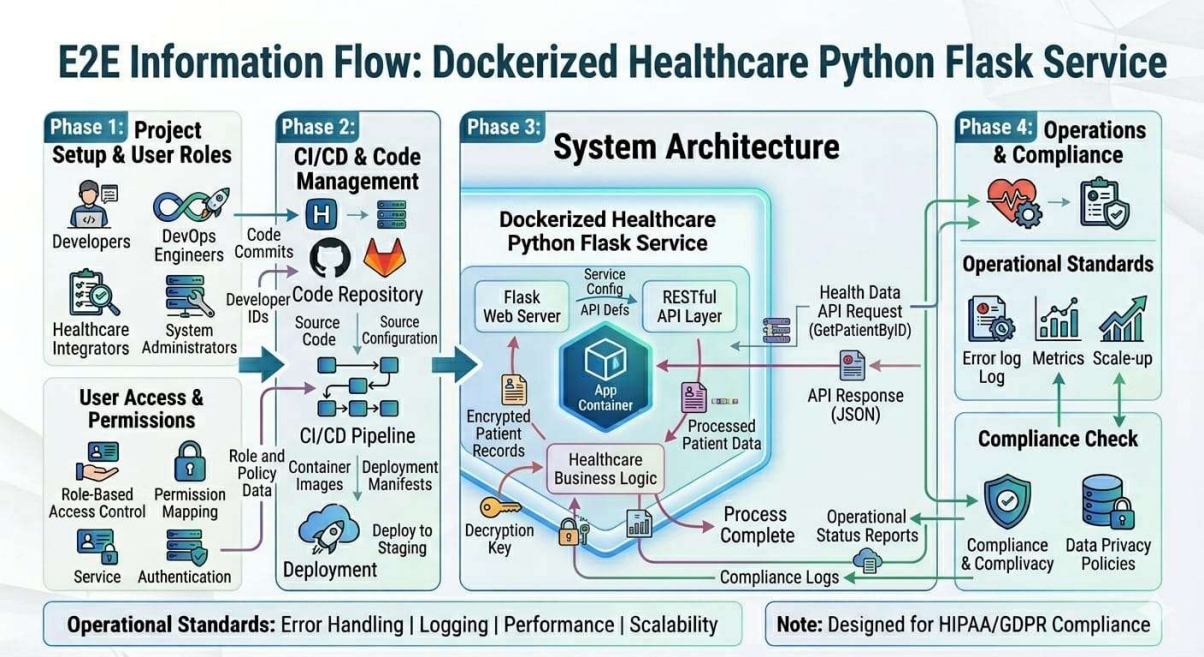
2.1. Application Design



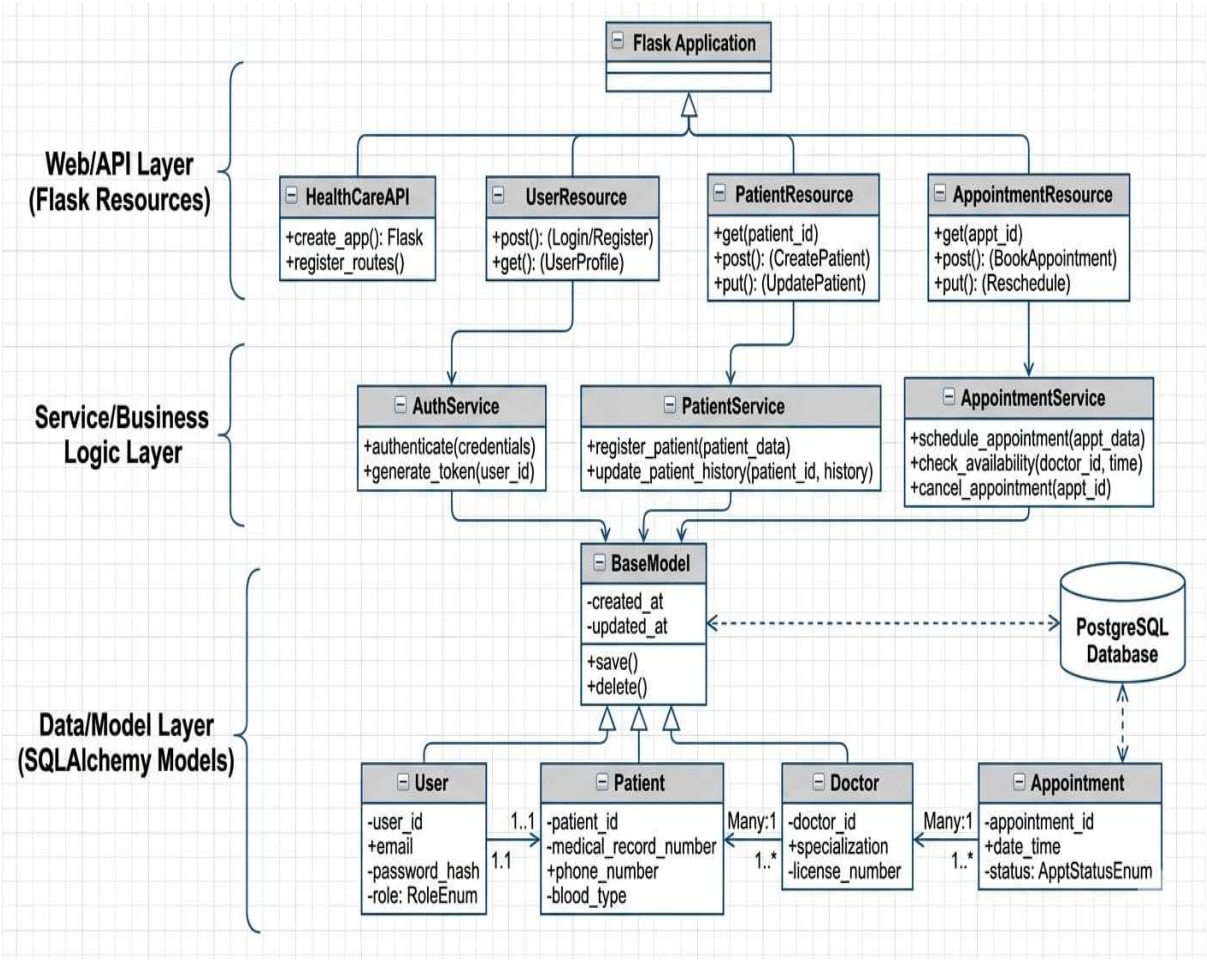
2.2. Process Flow



2.3. Information Flow



2.4. Class Diagram



2.5. Components Design

The technology stack used in the Dockerized Healthcare Flask Service includes:

Technology	Purpose
Python 3.x	Core programming language for backend logic and API development
Flask	Lightweight web framework used to build RESTful APIs
Docker	Containerization platform to package application and dependencies
Docker Compose	Tool to manage multi-container applications
Flask API Service	Handles HTTP requests, routes endpoints, and manages backend application logic
Docker Container	Packages the application with all dependencies and ensures consistent deployment across environments

2.6. Key Design Considerations

Consideration	Design Decision	Rationale
Containerization	Used Docker to package application and dependencies	Ensures consistency across development, testing, and production environments
Scalability	Designed stateless Flask APIs	Allows easy scaling using multiple containers
Performance	Optimized API response handling	Reduces latency and improves user experience
Error Handling	Implemented try-except blocks	Ensures system stability and graceful failure handling
Security	Input validation and environment variables	Protects against invalid data and secures sensitive configurations
Deployment	Used CI/CD tools (GitHub Actions/Jenkins)	Automates build, testing, and deployment process

3 . References

1. **Docker Documentation –** Containerization, images, and container management
2. **Flask Documentation –** Web framework setup and API development
3. **Docker Compose Documentation –** Multi-container application configuration

4. **GitHub Actions / Jenkins Documentation –** CI/CD pipeline setup and automation

5. **Nginx Documentation –** Reverse proxy and server configuration

6. **PostgreSQL / MySQL Documentation –** Database setup and management